Intro. to Python

Programming

Python

- Developed in 1989
- 2008: Python 3.0 was released
- Easy & Practical
- Popular language (broad community support – pypi.org)
- Free & Open source (Object Oriented Language)
  - Platform independent
  - Powerful
  - Fast



Supported Modes

- Normal
- Interactive

Applications

- Web development
- Game development
- Database interfaces
- Scientific/statistical analysis

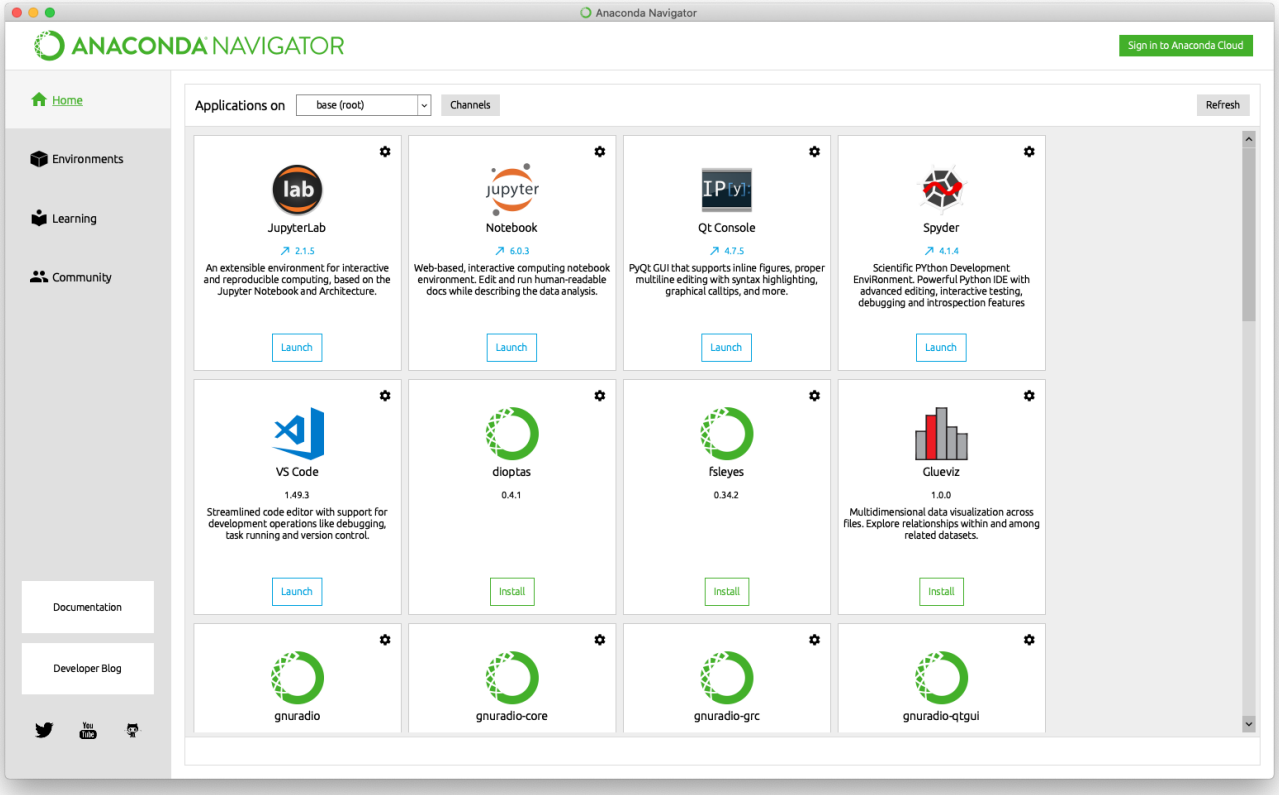
Who uses Python

- Education / Academia
- Professional Enterprise
- Hobbyist/Enthusiast

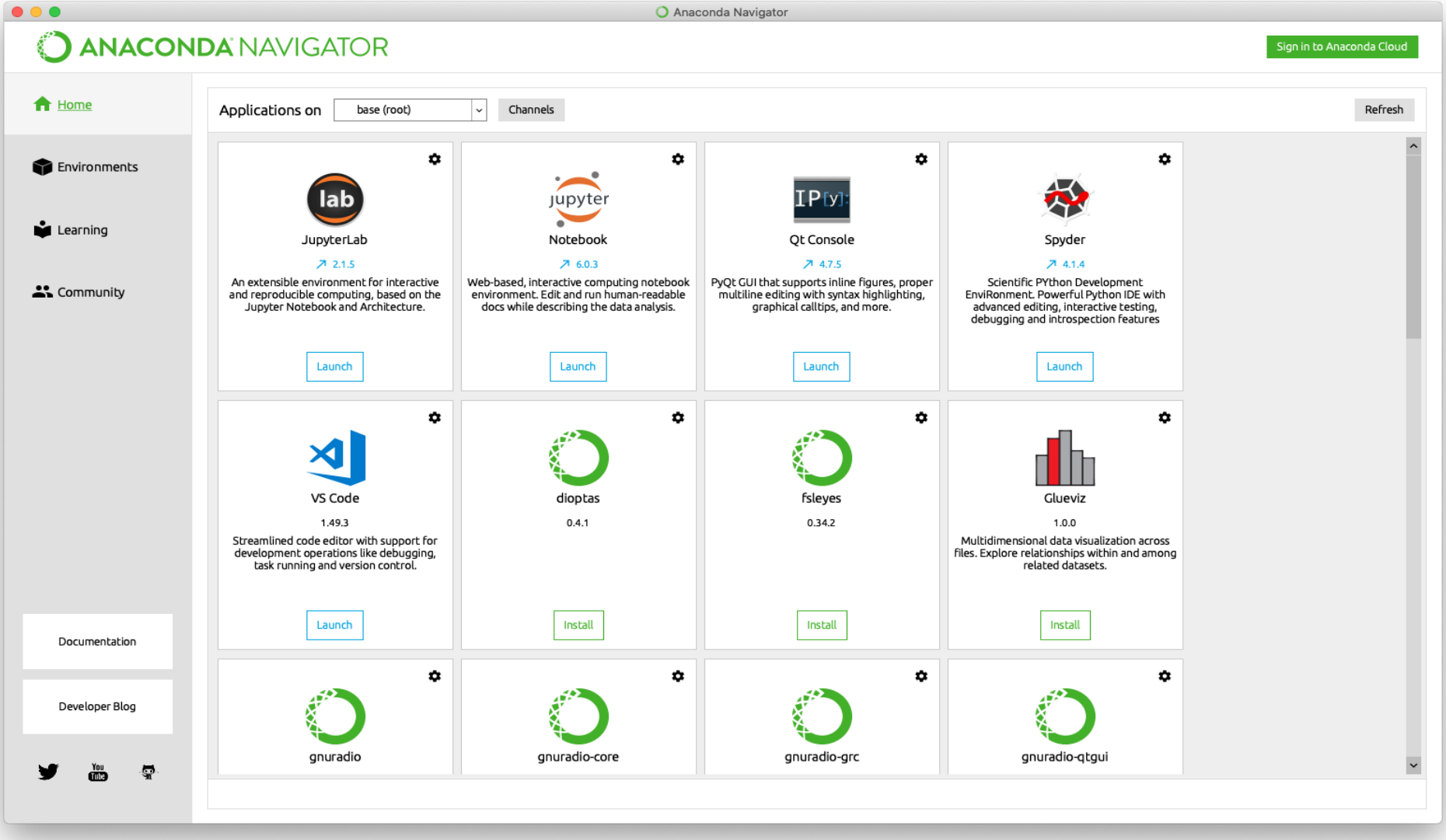
- Google
- Facebook
- Instagram
- Spotify
- Quora
- Netflix
- Dropbox
- Reddit

Programming IDEs / Tools

- Anaconda (Miniconda)
- IDEs
  - IDLE
  - Spyder
  - Jupyter Notebook
  - Google Colab
  - PyCharm



Installation (Anaconda)



History of Python



Guido van Rossum

Python 1: 1994

Python 2: 2000

Python 3: 2008

Programmers are more important than programs

“Hello World” in Java

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

Yuck

“Hello World” in C++

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello world!" << endl;
}
```

Double Yuck

“Hello World” in Python

```
print("Hello world!")
```

Python Basics



- Interactive Interpreter
- Comments
- Variables and Types
- Numbers and Booleans
- Strings and Lists
- Console I/O
- Control Flow
- Loops

Writing your 1st Program (Interactive Mode)

$ python -V

$ python 3.7.1

OR

$ python - -version

$ python 3.7.1

$ python

>>>

>>>print(“Hello World”)

Hello World

Writing your 1st Program (Interactive Mode)

```
>>> x = “Hello World”
>>> print(x)
Hello World
>>> quit()
$
```

Interactive Interpreter

```
sredmond$
```

Interactive Interpreter

```
sredmond$ python
```

Interactive Interpreter

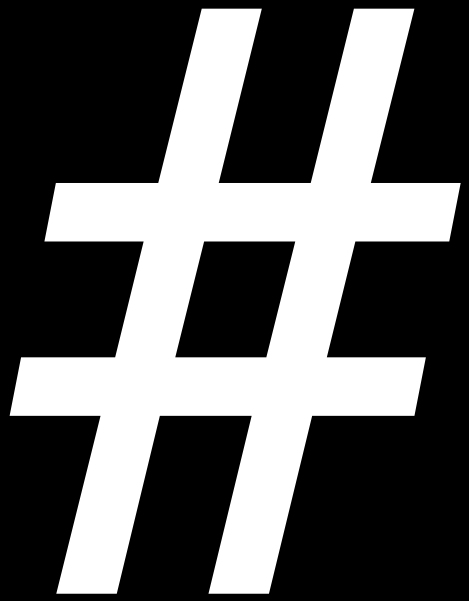
```
sredmond$ python
Python 3.4.3 (v3.4.3:9b73f1c3e601, Feb 23 2015, 02:52:03)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Interactive Interpreter

```
sredmond$ python
Python 3.4.3 (v3.4.3:9b73f1c3e601, Feb 23 2015, 02:52:03)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```



Hashtag



Sharp

Pound Sign

Number Sign

Octothorpe

Comments

Comments

```
# Single line comments start with a '#'
```

Comments

```
# Single line comments start with a '#'

"""
    Multiline strings can be written
    using three "s, and are often used
    as function and module comments
"""
```

Before jumping onto Programming

- print (“…….”)
- Assignment operator =
- Variables

- Operators
  - Arithmetic Operators (+, -, *, /, %, **, //)
  - Comparison (Relational) Operators (==, !=, >, <, >=, <=)
  - Assignment Operators (=, +=, -=, *=, /=, %=, **=, //=)
  - Logical Operators (and, or, not)
  - Bitwise Operators (&, |, ^, ~)
  - Membership Operators (in, not in)
  - Identity Operators (is, is not)

Symbol	Task Performed	Example	Result
+	Addition	4 + 3	7
-	Subtraction	4 - 3	1
/	Division	7 / 2	3.5
%	Mod	7 % 2	1
*	Multiplication	4 * 3	12
//	Floor division	7 // 2	3
**	Power of	7 ** 2	49

Operator	Output
x == y	True if x and y have the same value
x != y	True if x and y don't have the same value
x < y	True if x is less than y
x > y	True if x is more than y
x <= y	True if x is less than or equal to y
x >= y	True if x is more than or equal to y

Reserved/Keywords

- Words you cannot use as constants or variables or any other identifier names

and
as
assert
break
class
continue
def
del
elif
else
except
exec
finally
for
from
global
if
import
in
is
lambda
not
or
pass
print
raise
return
try
while
with
yield

Standard Data Types

- The data stored in memory can be of many types.
- For example, a person's age is stored as a numeric value and his or her address is stored as alphanumeric characters.

Python standard data types −

- Numbers
- String
- List
- Tuple
- Dictionary

Converting Values and Data Types

Converting Values and Data Types

```
str(42)          # => "42"
```

Converting Values and Data Types

```
str(42)          # => "42"
int("42")        # => 42
```

Converting Values and Data Types

```
str(42)          #  =>  "42"
int("42")        #  =>  42
float("2.5")     #  =>  2.5
```

Converting Values and Data Types

```
str(42)          #  =>  "42"
int("42")        #  =>  42
float("2.5")     #  =>  2.5
float("1")       #  =>  1.0
```

b = 125.0
c = 390.8

```
print(int(b))   #float() str()
print(int(c))
```

```
user = "Sammy"
lines = 50

print("Congratulations, " + user + "! You just wrote " + lines + " lines of code.")
```

Output

```
TypeError: Can't convert 'int' object to str implicitly
```

```
print("Congratulations, " + user + "! You just wrote " + str(lines) + " lines of code.")
```

Output

```
Congratulations, Sammy! You just wrote 50 lines of code.
```

Lines and Indentation

- Python does not use braces({}) to indicate blocks of code for class and function definitions or flow control.
- Blocks of code are denoted by line indentation, which is rigidly enforced.
- **Example:**

```
if True:
        print(“True”)
else:
        print(“False”)
```

Statements

- Complete sentence and command
- Statements in Python typically end with a new line

```
total = item_one + \    #Multiline Statements
        item_two + \
        item_three
```

- The statements contained within the [], {}, or () brackets do not need to use the line continuation character.
- print("'hello'");print("""world""");x="2018";print(x) **#Multiple statements on a single line**

```
========== RESTART: C:/Progs/comments.py =
'hello'
world
2018
>>>
```

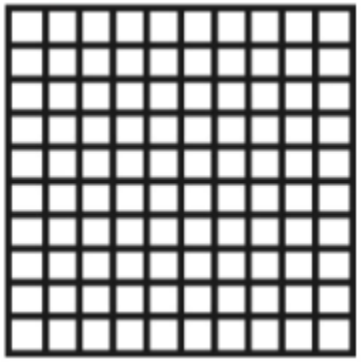
Escape Sequences

- \
- \n
- \t
- \b
- \’
- \u2192
- \\

Escape sequence	Meaning	Example	Output
`\`	Escape/line continuation depending on context	`print("Hello " \ "World")`	`Hello World`
`\n`	New line	`print("Hello\nWorld")`	`Hello` `World`
`\t`	Tab	`print("Name:\tSammy")`	`Name: Sammy`
`\b`	Backspace	`print("Hell\bo")`	`Helo` *
`\'`	Single quotation mark	`print('It\'s Python')`	`It's Python`
`\u2192`	Unicode character	`print("\u2192")`	→
`\\`	Backslash character	`print("C:\\Python")`	`C:\Python`

Memory Concepts - Variables

- Variable is a memory location
- Address in hexadecimal format
- Human readable are variables



Function Definition (Example)

```
DEF MULTIPLYTWONUMBERS(X,Y)
RESULT=X*Y
PRINT(RESULT)
RETURN
```

Variables

Variables

Variables

```
x = 2
```

No semicolon!

Variables

```
x = 2
x * 7
# => 14
```

No semicolon!

Variables

```
x = 2
x * 7
# => 14

x = "Hello, I'm "
```

No semicolon!

Variables

No semicolon!

```
x = 2
x * 7
# => 14

x = "Hello, I'm "
x + "Python!"
# => "Hello, I'm Python"
```

Variables

```
x = 2
x * 7
# => 14
```

No semicolon!

What happened here?!

```
x = "Hello, I'm "
x + "Python!"
# => "Hello, I'm Python"
```

Python Identifiers

- Name used to identify variable, function, class, object or module
- Starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).
- Python does not allow punctuation characters such as @, $, and % within identifiers.

Where's My Type?

In Java or C++

```
int x = 0;
```

Where's My Type?

In Python

```
x = 0
x = 'Fawad'
```

Where's My Type?

Variables in Python are **dynamically-typed**: declared without an explicit type

However, **objects** have a type, so Python knows the type of a variable, even if you don't

Strongly Typed

- Numeric
- String
- List
- Tuple
- Dictionary

Where's My Type?

Variables in Python are **dynamically-typed**: declared without an explicit type

However, **objects** have a type, so Python knows the type of a variable, even if you don't

```
type(1)          # => <class 'int'>
type("Hello")    # => <class 'str'>
type(None)       # => <class 'NoneType'>
```

Example

HelloWorld.py

```
1  HellowWorld = 'Hello World!'
2  print('HelloWorld')
```

```
Benjamins-Mac:03-Variables benjaminfinkel$ python HelloWorld.py
HelloWorld
```

HelloWorld.py

```
1  UserName = 'bfinkel'
2  FullName = 'Benjamin Finkel'
3
4
5  print(UserName)
6  print(FullName)
7
8  AltName = UserName
9  IncorrectName = 'FullName'
10
11 print(AltName)
12 print(IncorrectName)
```

```
Benjamins-Mac:03-Variables benjaminfinkel$ python HelloWorld.py
bfinkel
Benjamin Finkel
bfinkel
FullName
Benjamins-Mac:03-Variables benjaminfinkel$
```

HelloWorld.py

```
1   UserName = 'bfinkel'
2   FullName = 'Benjamin Finkel'
3
4
5   print(UserName)
6   print(FullName)
7
8   AltName = UserName
9   IncorrectName = 'FullName'
10  NewName = UserName = FullName
11  NewNAme = "New Name" = FullName
12  print(NewName)
13  print(UserName)
14
```

```
Benjamins-Mac:03-Variables benjaminfinkel$ python HelloWorld.py
  File "HelloWorld.py", line 11
    NewNAme = "New Name" = FullName
SyntaxError: can't assign to literal
Benjamins-Mac:03-Variables benjaminfinkel$
```

Numbers and Math

Numbers and Math

Numbers and Math

```
3               # =>  3    (int)
3.0             # =>  3.0  (float)
```

Python has 3 numeric types

`Int,float, complex`

int	float	complex
10	0.0	3.14j
100	15.20	45.j
-786	-21.9	9.322e-36j
080	32.3+e18	.876j
-0490	-90.	-.6545+0J
-0x260	-32.54e100	3e+26J
0x69	70.2-E12	4.53e-7j

Numbers and Math (Python 2.x)

```
3               #  =>  3    (int)
3.0             #  =>  3.0  (float)

1 + 1           #  =>  2
8 - 1           #  =>  7
10   * 2        #  =>  20
9 / 3           #  =>  3
5 / 2           #  =>  2
7 / 1.4         #  =>  5.0
```

Numbers and Math (Python 3.x)

```
7 // 3        # => 2  (integer division)
7 % 3        # => 1  (integer modulus)
2 ** 4       # => 16 (exponentiation)
```

```
>>> 7//3
2
>>> 7/3
2.3333333333333335
>>> 7.0//3
2.0
>>> 70/3
23.333333333333332
>>> 7.0//3
2.0
>>> 7.0/3
2.3333333333333335
>>>
```

HelloWorld.py

```
1  varString = "This is a string variable.  It can have " \
2              "any combination of letters, numbers, or " \
3              "special characters."
4
5  varInteger = 32
6
7  varLong = 12345L
8
9  varFloat = 1290.60
10
11 varList = [1, 2, 3.5, "Ben", "Alice", "Sam"]
12
13 varTuple = (4, 5, 6.5, "Alex", "Barbara", "Amy")
14
15 varDictionary = { 'First': 'The first item in the dictionary',
16                   'Second' : 'The second item in the dictionary' }
17
18 print(varString)
19 print(varInteger)
20 print(varLong)
21 print(varFloat)
22 print(varList)
23 print(varTuple)
24 print(varDictionary)
```

Booleans

Booleans

```
True                # => True
False               # => False
```

`bool` is a subtype of `int`, where `True == 1` and `False == 0`

Booleans

```
True              # => True
False             # => False


not True         #  => False
True or False     # => True (short-circuits)
True and False   #  => False
```

`bool` is a subtype of `int`, where `True == 1` and `False == 0`

Booleans

```
True              # => True
False             # => False

not True          # => False
True and False    # => False
True or False     # => True (short-circuits)


1  == 1           #  =>  True

2  * 3 ==  5      #  =>  False

1  != 1           #  =>  False

2  * 3 !=  5      #  =>  True
```

`bool` is a subtype of `int`, where `True == 1` and `False == 0`

Booleans

```
True              # => True
False             # => False

not True          # => False
True and False    # => False
True or False     # => True (short-circuits)
```

`bool` is a subtype of `int`, where `True == 1` and `False == 0`

```
1  == 1              #  =>  True
2  * 3 ==    5       #  =>  False
1  != 1              #  =>  False
2  * 3 !=    5       #  =>  True
1  < 10              #  =>  True
2  >= 0              #  =>  True
1  < 2 <   3         #  =>  True    (1  <  2  and  2  <  3)
1 < 2 >= 3          # => False (1 < 2 and 2 >= 3)
```

Strings

Strings

Strings

No `char` in Python!

Both `'` and `"` create string literals

Strings

No `char` in Python!

Both ' and " create string literals

```
greeting = 'Hello'
group = "world"
```

Strings

No `char` in Python!

Both ' and " create string literals

```
greeting = 'Hello'
group = "world"


greeting + ' ' + group + '!' # => 'Hello world!'
```

Indexing

```
  0 1 2 3 4 5
s='Arthur'
```

Indexing

```
  0 1 2 3 4 5
s='Arthur'
```

```
s[0] == 'A'
s[1] == 'r'
s[4] == 'u'
s[6] # Bad!
```

Negative Indexing

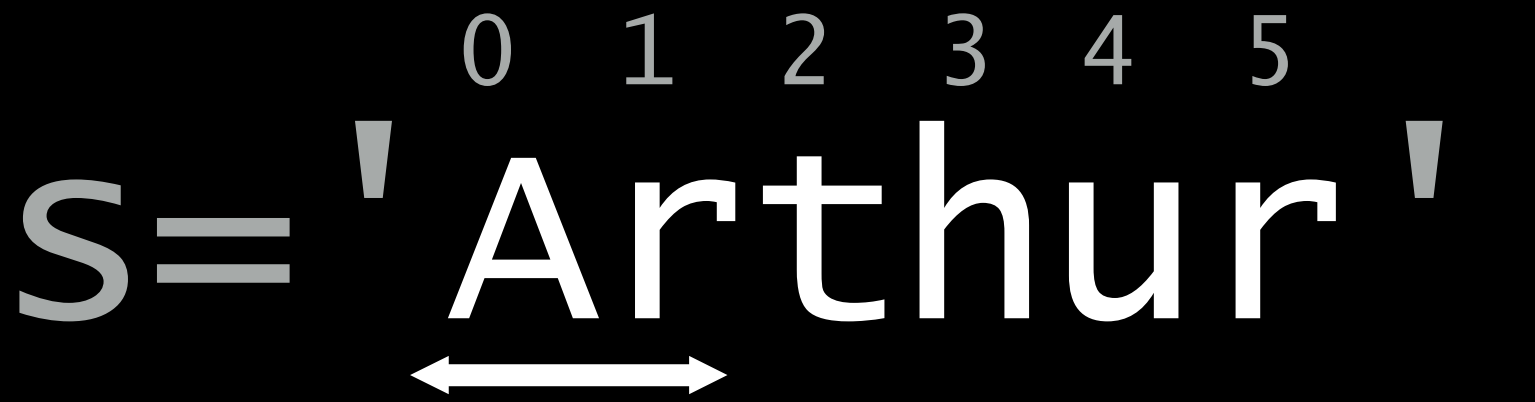
```
  0  1  2  3  4  5
s='Arthur'
 -6 -5 -4 -3 -2 -1
```

```
s[-1] == 'r'
s[-2] == 'u'
s[-4] == 't'
s[-6] == 'A'
```

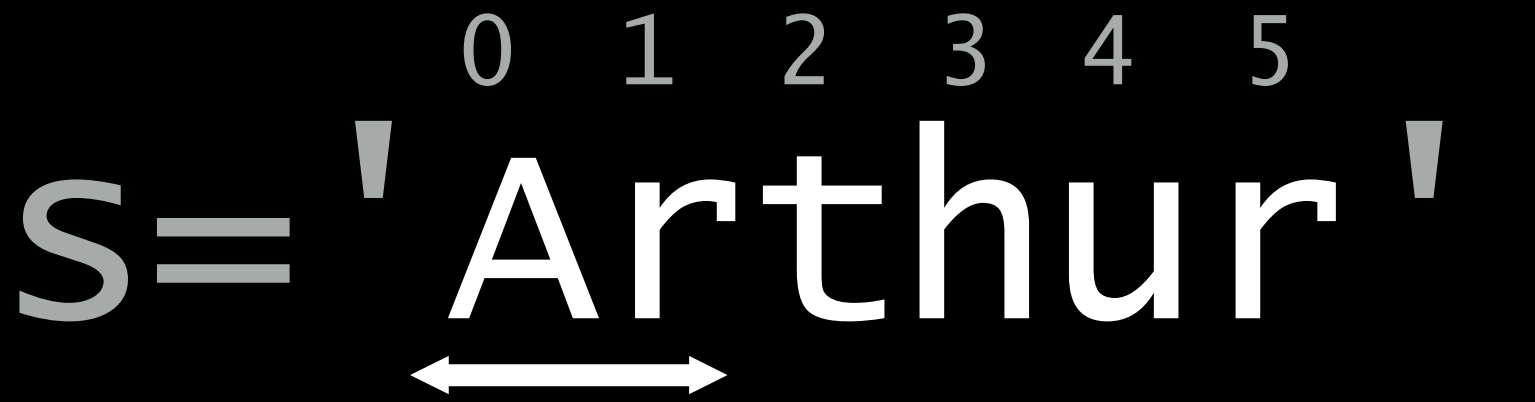
Slicing

```
  0 1 2 3 4 5
s='Arthur'
```

Slicing

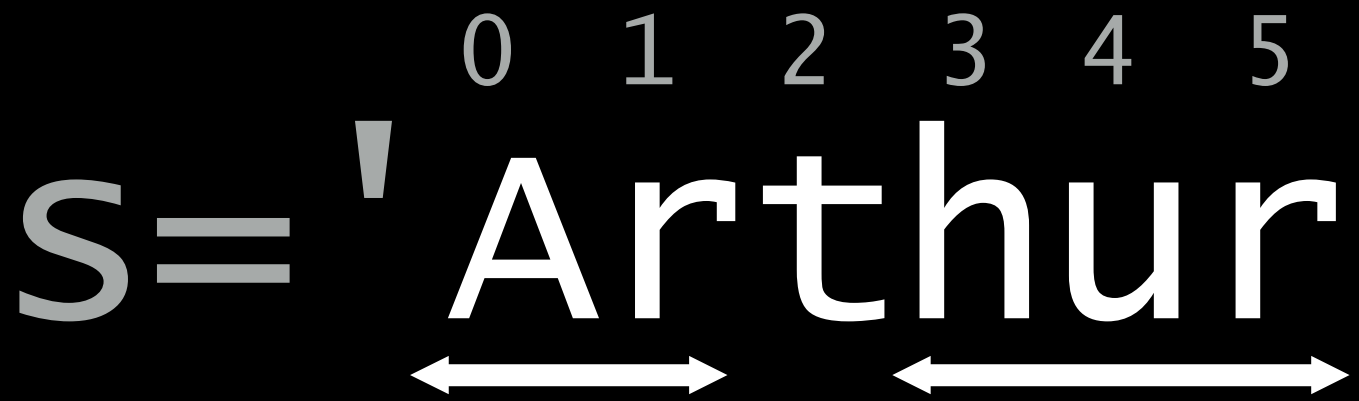


Slicing



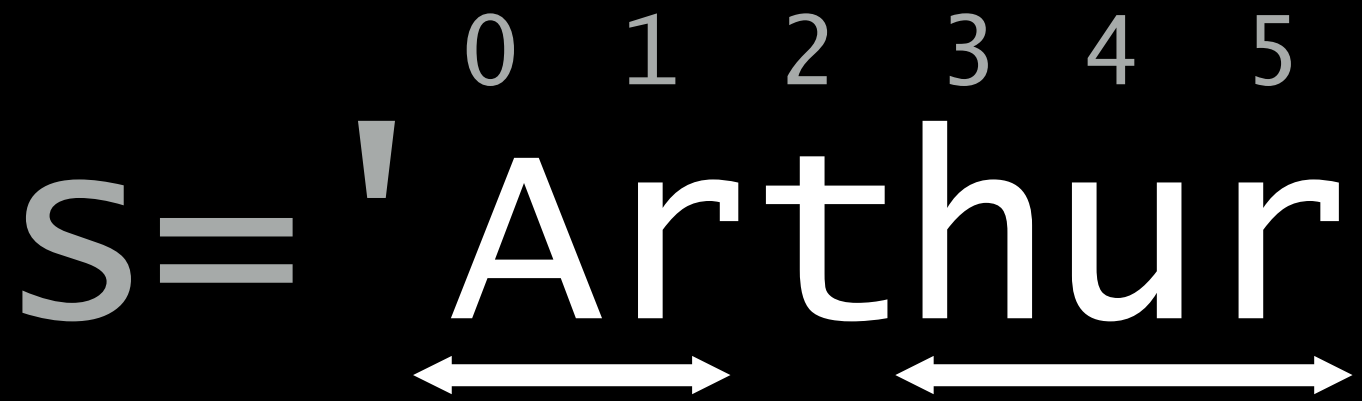

```
s[0:2] ==  'Ar'
```

Slicing



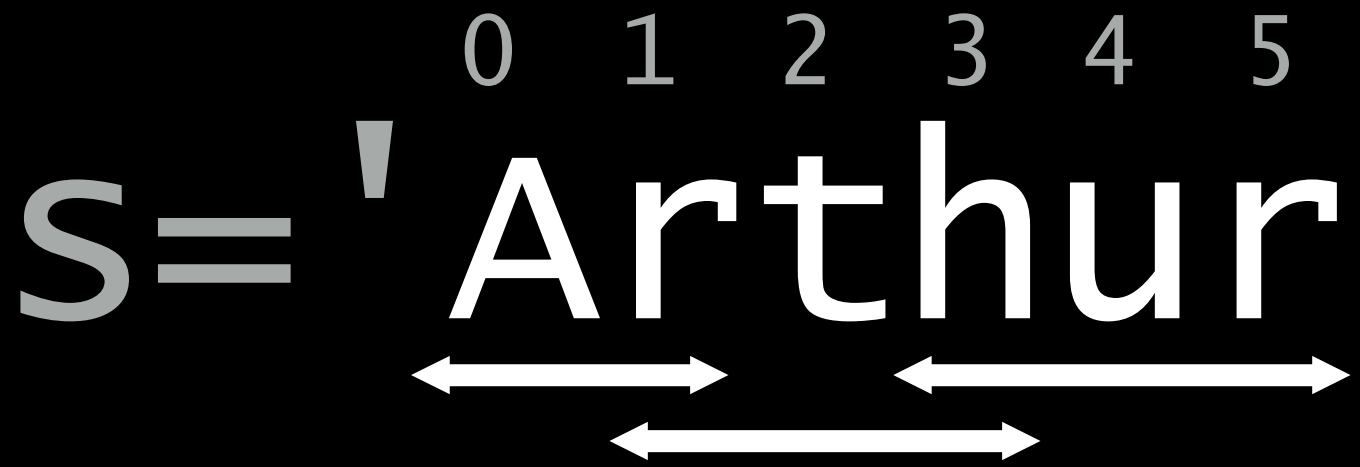

```
s[0:2] ==  'Ar'
```

Slicing



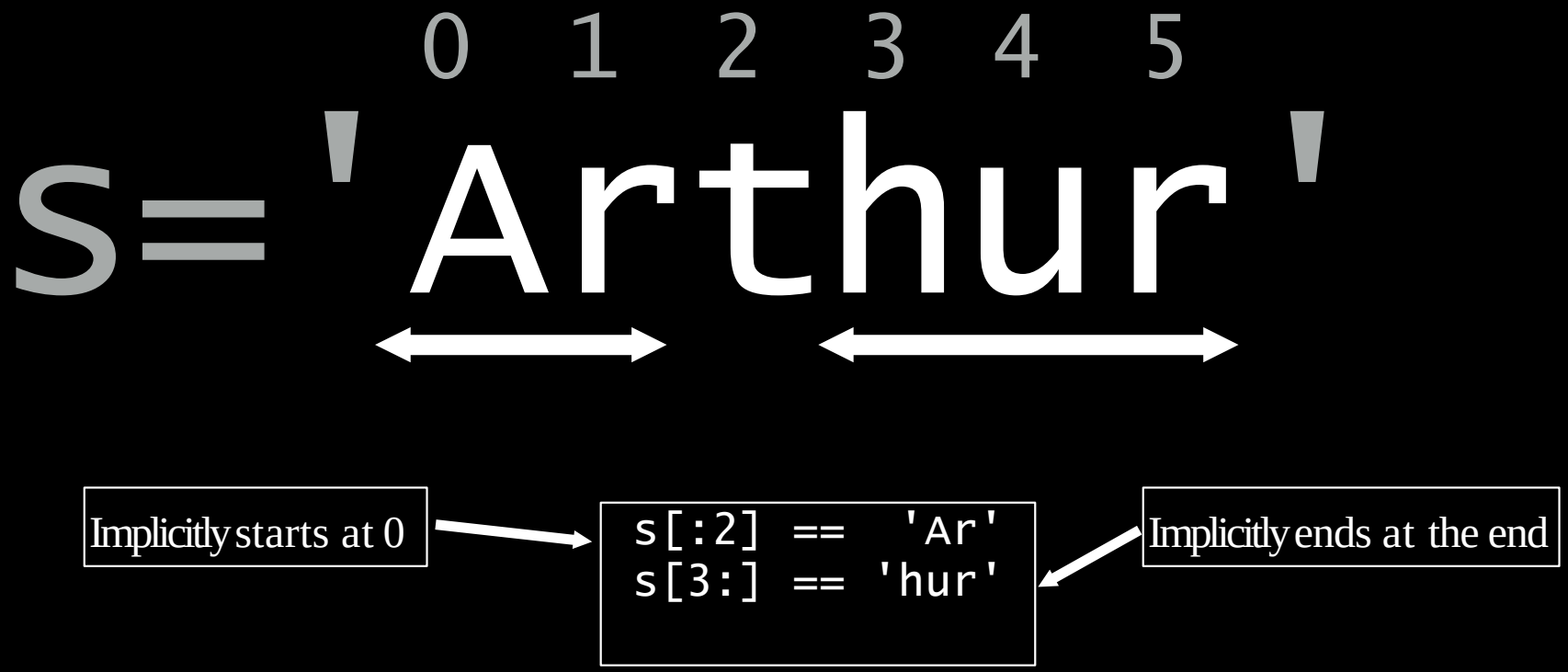

```
s[0:2] ==  'Ar'
s[3:6] == 'hur'
```

Slicing




```
s[0:2]  ==  'Ar'
s[3:6]  == 'hur'
s[1:4]  == 'rth'
```

Strings


0 1 2 3 4 5
s='Arthur'
Implicitly starts at 0
s[:2] == 'Ar'
s[3:] == 'hur'
Implicitly ends at the end

Strings

```
  0 1 2 3 4 5
s='Arthur'
```

Strings

```
  0 1 2 3 4 5
s='Arthur'
```

Can also pass a step size

```
s[1:5:2] == 'rh'
s[4::-2] == 'utA'
```

Strings

```
  0 1 2 3 4 5
s='Arthur'
```

Can also pass a step size

```
s[1:5:2] == 'rh'
s[4::-2] == 'utA'
```

Reversing a string

```
s[::-1] == 'ruhtrA'
```